

Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAXN 100005
#define MAXM 100005

typedef unsigned long long ull;

int parent_node[MAXN];

int find_set(int v) {
    if (v == parent_node[v])
        return v;
    return parent_node[v] = find_set(parent_node[v]);
}

int union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b) {
        parent_node[b] = a;
        return 1;
    }
    return 0;
}

typedef struct {
    int u, v;
    ull mask;
} Road;

Road roads[MAXM];
ull c[65];

int main() {
    int n, m, k;
    if (scanf("%d %d %d", &n, &m, &k) != 3) return 0;

    for (int i = 1; i <= k; ++i) {
        scanf("%llu", &c[i]);
```

```

}

for (int i = 0; i < m; ++i) {
    int u, v, l;
    scanf("%d %d %d", &u, &v, &l);
    ull mask = 0;
    for (int j = 0; j < l; ++j) {
        int token;
        scanf("%d", &token);
        if (token <= 62) {
            mask |= (1ULL << (token - 1));
        }
    }
    roads[i].u = u;
    roads[i].v = v;
    roads[i].mask = mask;
}

int i;
for(i=1;i<=n;++i) {
    parent_node[i] = i;
}

int comps = n;
for (int j = 0; j < m; ++j) {
    if (union_sets(roads[j].u, roads[j].v)) {
        comps--;
    }
}

if (comps > 1) {
    printf("-1\n");
    return 0;
}

ull selected_mask = 0;
for (int bit = k - 1; bit >= 0; --bit) {
    for (i = 1; i <= n; ++i) parent_node[i] = i;
    comps = n;

    for (int j = 0; j < m; ++j) {
        ull test_mask = selected_mask | ((1ULL << bit) - 1);
        if ((roads[j].mask & ~test_mask) == 0) {
            if (union_sets(roads[j].u, roads[j].v)) {

```

```
        comps--;  
    }  
}  
  
if (comps > 1) {  
    selected_mask |= (1ULL << bit);  
}  
  
ull total_cost = 0;  
for (int bit = 0; bit < k; ++bit) {  
    if ((selected_mask >> bit) & 1ULL) {  
        total_cost += c[bit + 1];  
    }  
}  
  
printf("%llu\n", total_cost);  
return 0;  
}
```

Input:

```
3 3 4  
1 2 5 10  
1 2 2 1 2  
1 3 1 3  
2 3 1 4
```

Output:

```
8
```